

BIÊN BẢN HỌP KỸ THUẬT & KẾ HOẠCH TRIỂN KHAI

Tổng hợp định hướng kiến trúc AgentCore/Bedrock, quy chuẩn QA/Testing và lộ trình tích hợp Security & EVAL vào TI

NGÀY GHI TRỌNG TÂM KỸ THUẬT

NHẬN

28/09/2026(Ragas + ML), TI

PHẠM VI ÁP DỤNG

Kế hoạch công việc tuần (Thứ

3 – Thứ 6)

NGƯỜI PHÊ DUYỆT

CUỐI

Human-in-the-Loop

(Report)

PHẦN I ĐỊNH HƯỚNG KIẾN TRÚC AI, HẠ TẦNG THỰC THI & LƯỒNG NGHIỆP VỤ

1. Nền tảng Model & AgentCore

Core AI

- Huấn luyện (Training):** Các model phục vụ training thống nhất định hướng sử dụng trên nền tảng **AWS Bedrock**.
- Khởi tạo Custom Agent:** Sử dụng **AgentCore** để sinh ra các agent tùy chỉnh (custom agents) theo từng tác vụ.
- Đặc tả Agent:** Mỗi agent khi thiết kế bắt buộc phải xác định rõ: sử dụng **LLM model** nào kết hợp với danh sách **những công cụ (tools)** nào mà agent đó được phép gọi.
- Công cụ AI hỗ trợ:** Sử dụng **AI Freebuff** để khai thác và vận hành các tác vụ AI trong quá trình làm việc.

2. Hạ tầng Fargate, Chi phí & Khả năng Scale

Hạ tầng & Scale

- Độ trễ trên AWS Fargate:** Hiện tại việc chạy qua Fargate tốn nhiều thời gian khởi tạo/xử lý, mất khoảng **2 – 3 phút**.
- Quan điểm về Chi phí:** Hiện tại **chưa đặt giới hạn chi phí** vì mục tiêu trọng tâm là hướng tới **scale lâu dài**. Tạm thời *khoan quan tâm* tới bài toán giả định mức chi phí A hay B sẽ giới hạn tới đâu.
- Tái cấu trúc Agent:** Các agent hiện tại chỉ mới đáp ứng theo kiểu *"mì ăn liền"* (tạm thời/ngắn hạn). Khi mở rộng quy mô (scale lên) sẽ không đảm bảo ổn định, bắt buộc **cần phải tái tạo (refactor/rebuild)** lại.

3. Kiểm soát Vòng lặp (Harness & Guardrails)

An toàn Thực thi

- Cơ chế Vòng lặp (Loop):** Sử dụng **Harness** cho phép hệ thống thực hiện chạy vòng lặp (loop) tự động.
- Rủi ro bị khóa (Ban):** Hiện tại **chưa có cơ chế giới hạn số lần lặp (limit loop)**, nếu để chạy tự do rất dễ dẫn đến gọi quá tải và **bị ban**.
- Đề xuất kiểm soát:** Đã có đề xuất bổ sung **AgentCore Harness: TIJobRunner** và **Token Budget Guard** (giới hạn lượng token/run), nhưng mới ở mức ý tưởng, **chưa nêu thông số thiết kế cụ thể**.

4. Đánh giá, Luồng Artifact & Jev AI

Business Flow

- Bedrock Evaluation:** Hiện chỉ đứng ở khúc cuối (phía end-user), đánh giá chung là **chưa đạt yêu cầu (chưa ok)**.
- Kết quả & Quyền quyết định:** Đầu ra cuối cùng của hệ thống là **Report (Báo cáo)**; người đưa ra quyết định cuối cùng là **con người (Human)**.
- Luồng Artifact động (Dynamic):** Toàn bộ business flow đều đi dựa theo **Artifact**, do đó kiến trúc phải có tính **dynamic** (linh hoạt theo từng thời điểm/ngữ cảnh) thì mới thích nghi và "sinh tồn" được với mọi kịch bản.
- Vai trò Jev AI:** Chỉ đóng vai trò là AI **chấm điểm (scoring)** để điều phối chạy luồng nào, hoàn toàn không thực hiện suy luận (reasoning).

Tóm tắt Yêu cầu Cốt lõi về Kiến trúc Hệ thống (Key Architectural Takeaways)

1. Tư duy Kiểm thử & Phản biện (Testing Mindset)

- Thực trạng:** Hiện tại khi thực hiện kiểm thử, các thành viên chỉ dừng ở việc chạy test và đăng lỗi (post bug) một cách thụ động, **không ai đặt câu hỏi**.
- Yêu cầu bắt buộc:** Quá trình Test **cần phải có Question (đặt câu hỏi)** để làm rõ logic nghiệp vụ, nguyên nhân gốc rễ và các kịch bản biên.
- Đồng bộ phong cách (Style):** Khắc phục tình trạng hiện tại mỗi thành viên viết report một kiểu rời rạc, thiếu tính hệ thống.

2. Quy chuẩn Viết Bug Report (Bằng Tiếng Anh)

Viết bằng **Tiếng Anh (English)** ngắn gọn, rõ ràng để người đọc nhìn vào biết ngay **bug gì, ở đâu và sửa ở đâu**:

Header Note: [Bug Level] + [Bug Type] + [Page / Screen]

Expected Result: Tách thành mục riêng biệt (kết quả kỳ vọng).

Actual Result: Tách thành mục riêng biệt (lỗi thực tế xảy ra).

*** Lưu ý:** Tuyệt đối tách riêng Expected và Actual, không gom chung gây khó đọc và khó xử lý.

Phương án làm việc khi chưa truy cập được Git & Công cụ Hỗ trợ

- Tiếp cận qua gói mã nguồn (.zip):** Trong trường hợp chưa vào được kho mã nguồn Git, nhiệm vụ trọng tâm là giải nén và đọc hiểu trực tiếp cấu trúc bên trong file .zip.
- Khai thác công cụ AI:** Sử dụng con **AI Freebuff** để hỗ trợ phân tích, đọc hiểu mã nguồn và xử lý các tác vụ AI trong suốt quá trình làm việc.

1. Phân hệ Security (Pentest)

Chưa nối vào TI

- Nhận diện trong gói .zip:** Module Pentest bên trong file nén chính là phân hệ **Security**.
- Trạng thái triển khai:** Đã được dựng và đưa lên nhưng **chưa được tích hợp vào con TI**.
- Đánh giá kiến trúc:** Kiến trúc hiện tại của con Security còn lỏng lẻo ("hơi lỏ"), cần chủ động **sửa và tối ưu lại kiến trúc** (thực hiện trong Thứ 3 & Thứ 4).

2. Phân hệ EVAL (Ragas + ML)

Thiếu Test ML

- Cơ chế hoạt động hiện tại:** Sử dụng Ragas để test ở phía **end-user** nhằm cover trường hợp người dùng không vào được backend mà bắt buộc phải dùng **Chat**.
- Điểm khuyết thiếu:** Ragas hiện tại **đang thiếu phần kiểm thử Machine Learning (test ML)**; kiến trúc con EVAL cũng còn yếu ("hơi lỏ").
- Định hướng xử lý:** Sửa lại kiến trúc con EVAL, **tích hợp thêm phần ML vào con EVAL** (Thứ 5), sau đó nối con EVAL vào con TI.

Mục tiêu Kết nối Tổng thể vào Hệ thống TI (Target Integration Goal)

- Đọc hiểu toàn diện 2 phân hệ:** Nắm chắc luồng dữ liệu, cách tạo và truyền **Artifact** của cả con **Security (Pentest)** và con **EVAL (Ragas + ML)** sau khi đã chỉnh sửa kiến trúc.
- Hợp nhất vào TI & Hướng tới Scale:** Từ những gì đã hiểu về 2 con đó, tiến hành vẽ kiến trúc mới tích hợp trực tiếp vào con **TI hiện tại**, đồng thời giải quyết bài toán mở rộng quy mô (scale) lâu dài cho toàn bộ hệ thống TI.

PHẦN IV

KẾ HOẠCH HÀNH ĐỘNG & LỊCH TRÌNH CÔNG VIỆC TRONG TUẦN

Mốc Thời Gian	Hạng Mục / Phân Hệ	Nhiệm Vụ Chi Tiết Cần Thực Hiện	Đầu Ra Dự Kiến
Nhiệm vụ Nền tảng (Xuyên suốt tuần)	Đọc hiểu 2 con Security & EVAL (Qua gói .zip)	- Giải nén gói .zip (khi chưa vào được Git), đọc hiểu sâu toàn bộ cấu trúc, luồng xử lý và Artifact của 2 con Security (Pentest) và EVAL (Ragas). - Sử dụng AI Freebuff để hỗ trợ làm việc với AI. - Áp dụng quy chuẩn QA mới: Đặt câu hỏi khi test và viết Bug Report bằng tiếng Anh (tách rõ <i>Expected</i> và <i>Actual Result</i>).	Nắm vững kiến trúc, luồng hoạt động & điểm nghẽn của 2 con
Thứ Ba & Thứ Tư (29/09 – 30/09/2026)	Con Security (Pentest) Đọc hiểu & Sửa kiến trúc	- Đọc hiểu chi tiết luồng Pentest của con Security (hiện đã lên nhưng chưa tích hợp vào TI). - Đánh giá các điểm bất cập trong kiến trúc hiện tại đang còn "hơi lỏng". Chủ động sửa đổi và tối ưu lại kiến trúc con Security để chuẩn bị tích hợp vào hệ thống TI.	Kiến trúc Security (Pentest) đã được sửa đổi hoàn chỉnh
Thứ Năm (01/10/2026)	Con EVAL (Ragas + ML) Sửa kiến trúc & Bổ sung ML	- Đọc hiểu và sửa lại kiến trúc đang còn "hơi lỏng" của con EVAL (đang dùng Ragas test end-user qua Chat khi không vào được backend). Tích hợp bổ sung phần kiểm thử Machine Learning (test ML) vào con EVAL để bù đắp phần thiếu hụt của Ragas. - Chuẩn hóa đầu ra của con EVAL để sẵn sàng nối vào con TI.	Con EVAL hoàn thiện kiến trúc mới + Tích hợp xong Test ML
Thứ Sáu (02/10/2026)	Vẽ Kiến trúc Mới tích hợp vào TI & Phương án Scale TI	- Từ những gì đã hiểu về 2 con (Security & EVAL): Thiết kế và vẽ 01 Kiến trúc Mới tích hợp cả 2 con vào hệ thống TI hiện tại. Tính toán phương án Scale hệ thống TI lâu dài: - Tái tạo các agent từ kiểu "mì ăn liền" sang kiến trúc chuẩn trên Bedrock & AgentCore (xác định rõ LLM model + Tools). - Đảm bảo luồng nghiệp vụ đi theo Artifact động, dùng Jev AI chấm điểm điều phối và xuất Report cuối cho Human. - Giải quyết độ trễ 2–3 phút của Fargate; cụ thể hóa AgentCore Harness: TIJobRunner (giới hạn vòng lặp tránh bị ban) & Token Budget Guard (giới hạn token/run).	Bản vẽ Kiến trúc Mới tích hợp vào TI & Phương án Scale hệ thống TI